
Analisis de codigo recibido y pre-auditoría de Seguridad

Fecha: 19 de Enero, 2026

Objetivo: Evaluar el estado actual del código legado (`LegacyCod/Materia.py`) para evaluar su continuidad, modificación o reestructuración.

Entorno de Hardware

La aplicacion corre sobre un entorno Linux, con la siguiente configuracion:

- Seeed Studio Edgebox RPi-200 4G eMMC 16GB (<https://www.seeedstudio.com/EdgeBox-RPi-200-CM4104016-p-5486.html>)
- LTE Cat 4 EC25-AUXGR-mini-PCle (<https://www.seeedstudio.com/LTE-Cat-4-EC25-AUXGR-mini-PCle-p-5885.html>)

Modo de uso

Con pantalla local:

```
DISPLAY=:0 python3 Materia.py
```

Sin pantalla local:

```
python3 Materia.py
```

1. Resumen Ejecutivo

El código actual presenta deudas técnicas críticas que comprometen la **estabilidad**, **seguridad** y **portabilidad** de la aplicación. La aplicación es un monolito rígido diseñado para un entorno Linux específico. La gestión incorrecta de la concurrencia y los recursos del sistema (puertos seriales, hilos) representa un riesgo alto de fallos silenciosos, bloqueos de interfaz y potencial corrupción de datos.

2. Defectos Críticos de Arquitectura

2.1. Monolito "Omnisciente" (`God Class`)

- **Hallazgo:** La clase `BásculaApp` (aprox. 870 líneas) viola el Principio de Responsabilidad Única (SRP).
- **Evidencia:** Esta única clase maneja simultáneamente:
 - Interfaz Gráfica (GTK/PyGObject).
 - Lógica de Negocio (Cálculo de porcentajes, validación de tolerancias).
 - Persistencia de Datos (SQL crudo mezclado con lógica de UI).
 - Comunicación Hardware (Puerto Serial Báscula y Módem GSM - Comandos AT).
 - Gestión de Hilos y Temporizadores (Lógica de envío programado).
- **Impacto:** Imposible de testear unitariamente (Unit Testing) sin hardware conectado o interfaz gráfica levantada. Cualquier cambio en la UI puede romper la lógica de negocio y viceversa. Aumenta drásticamente el costo (dinero y tiempo) de mantenimiento.

2.2. Configuración de hardware actual (Vendor Lock-in)

- **Hallazgo:** Rutas de dispositivos "hardcodeadas" y específicas para un sistema en particular.
- **Evidencia:**
 - Línea 22: `SMS_PORT = "/dev/ttyUSB2"`
 - Línea 518: `port_name = "/dev/ttyACM1"`
- **Impacto:** La aplicación **NO** funcionará si se realiza un cambio de hardware. Por ejemplo, si se cambia el puerto de la báscula o el módem, si se realiza una actualización del hardware donde esta corriendo la aplicación. Requiere reescritura para detectar puertos dinámicamente o utilizar un archivo de configuración externo.

2.3. Gestión de Concurrency Defectuosa

- **Hallazgo:** Fuga de memoria (Memory Leak) y condiciones de carrera.
- **Evidencia:**
 - Línea 365: `_sms_threads.append(t)`: La lista global de hilos crece indefinidamente con cada envío automático y nunca se limpia. En una operación continua, esto degradará el rendimiento hasta fallar.

- Línea 73-133: `enviar_sms_pdu_core` realiza múltiples llamadas con `time.sleep` dentro de hilos gestionados manualmente sin mecanismos robustos de parada o limpieza (`join`).
- Uso de `global _app_running` como única bandera de control, inseguro para aplicaciones complejas.
- Línea 866: Cierre abrupto con `os._exit(0)` , lo que impide que los bloques `finally` o destructores limpien recursos (cerrar conexiones DB, liberar handles de archivos), aumentando el riesgo de corrupción.

3. Auditoría de Seguridad

3.1. Integridad de Datos (Backup Inseguro)

- **Hallazgo:** Copia de seguridad de base de datos no atómica (Race Condition).
- **Evidencia:** Línea 832: Uso de `shutil.copy2(DB_NAME, ...)` mientras la base de datos está abierta y en uso por la aplicación.
- **Impacto:** Si se realiza un backup mientras se escribe un registro, el archivo resultante puede estar corrupto.
- **Solución:** Usar la API de Backup Online de SQLite.

3.1.1. Fragmentación de Datos (Inconsistencia de Persistencia)

- **Hallazgo:** La "Capa de Persistencia" es incoherente y está dispersa en múltiples formatos y archivos.
- **Evidencia:**
 - **SQLite (`Registro.db`):** Almacena los pesajes (Transaccional).
 - **TXT (`horas.txt` , `contactos.txt`):** Almacena configuración crítica (Horarios de envío, Números de teléfono).
 - **CSV (`Haciendas.csv`):** Almacena datos maestros (Listas de Haciendas y Suertes).
- **Impacto (Integridad Referencial):** No existe relación real entre los datos. Si se borra una Hacienda del CSV, los registros históricos en SQLite quedan huérfanos o con datos inválidos.
- **Impacto (Seguridad/Backup):** Realizar una copia de seguridad implica gestionar 4 archivos distintos. Es fácil olvidar uno (ej. guardar la BD pero perder la configuración de contactos) en una recuperación de desastres.

- **Solución:** Unificar **TODO** (Configuración, Maestros y Transacciones) dentro de la base de datos SQLite relacional.

3.2. Disponibilidad (UI Blocking / DoS)

- **Hallazgo:** Operaciones de I/O bloqueantes en el hilo principal.
- **Evidencia:** Aunque el SMS va en un hilo, la lectura de la báscula (`leer_peso_serial` , l. 517) tiene un `timeout=1` y se llama desde el hilo principal o callbacks de UI. Si el dispositivo no responde instantáneamente, la interfaz gráfica se congelará ("No responde") durante 1 segundo repetidamente, degradando la experiencia del usuario.

3.3. Validación de Entradas ()

- **Hallazgo:** Validación débil de tramas seriales.
- **Evidencia:** `leer_peso_serial` confía en la posición fija de los datos (`partes[2]`) tras un split simple. Si la báscula envía ruido o una trama parcial (común en serial), la conversión fallará o procesará datos erróneos.
- **Positivo:** El uso de parámetros `?` en las consultas SQL (l. 457, 611) protege contra Inyección SQL básica.

3.4. Seguridad de Dependencias

- **Protocolos:** La implementación manual de la construcción de tramas PDU para SMS ("reinventar la rueda") es propensa a errores de encoding (UCS2) y vulnerabilidades si se procesan caracteres maliciosos o inesperados.
- **Librerías:** El código depende de `gi` (GTK3). GTK3 es estable pero antiguo (GTK4 se lanzó en diciembre de 2020, después de estar en pruebas desde el 2016).

3.5. Falta de Control de Acceso y Gestión de Datos

- **Control de Acceso Inexistente:**
 - **Hallazgo:** La aplicación no cuenta con ningún sistema de login, usuarios o roles.
 - **Evidencia:** El método `__init__` (líneas 140-319) carga la interfaz y concede acceso total inmediatamente sin solicitar credenciales.
 - **Impacto:** Cualquier persona con acceso físico a la terminal puede operar la báscula, alterar configuraciones críticas (contactos, horas) y acceder a datos históricos.
- **Borrado de Datos No Protegido:**

- **Hallazgo:** La funcionalidad de "Limpiar base de datos" es accesible para cualquier usuario y carece de mecanismos de seguridad robustos.
- **Evidencia:** El método `on_clear_db` (línea 838) solo presenta un diálogo de confirmación simple (`Gtk.MessageDialog`). No requiere contraseña de supervisor ni doble confirmación.
- **Impacto:** Un usuario malintencionado o inexperto puede borrar permanentemente todo el historial de pesajes ("DELETE FROM registro") con solo dos clics, causando pérdida irreparable de información operativa.

3.6. Continuidad Operativa y Flexibilidad

- **Restricción Rígida de Entrada (Candado de Hardware):**

- **Hallazgo:** Los campos de peso están configurados estrictamente como "solo lectura" (`set_editable(False)`), obligando a que el dato provenga del puerto serial.
- **Aspecto Positivo:** En operación normal, actúa como un control de integridad eficaz, evitando errores de digitación o manipulación por parte del operador.
- **Defecto Crítico:** La aplicación carece de un **plan de contingencia (Bypass)**. Si falla el hardware periférico (cables rotos, conversores USB dañados, descalibración temporal), la operación se detiene totalmente (Denegación de Servicio operativa por un "Single Point of Failure").
- **Recomendación:** Implementar un modo de "Entrada Manual de Contingencia" que permita digitar el peso. Este modo debe ser **auditable** (registrar que se usó contingencia) y estar protegido por credenciales de **Supervisor**.

3.7. Análisis de Experiencia de Usuario (UI/UX)

- **Edición de Configuración "Primitiva":**

- **Hallazgo:** La gestión de contactos y horarios no usa formularios, sino que abre un editor de texto plano dentro de la aplicación (`_editar_archivo_texto`).
- **Defecto:**
 - **Sin Validación:** El usuario puede ingresar texto basura, formatos inválidos o borrar accidentalmente todo el archivo.
 - **Falta de Contexto:** La lista de teléfonos (`contactos.txt`) es solo una lista de números. No permite asociar un **Nombre** (e.g., "Jefe de Zona Norte"), obligando al operador a memorizar a quién pertenece cada número.

- **Datos Maestros Cripticos (Códigos vs. Nombres):**

- **Hallazgo:** Las listas desplegables de "Hacienda" y "Suerte" muestran únicamente códigos numéricos o alfanuméricos extraídos del CSV.
- **Defecto:** Aumenta drásticamente la carga cognitiva y el error humano. Seleccionar el código "4584" es propenso a errores; seleccionar "4584 - Finca La Esperanza" es seguro. El sistema carece de descripciones legibles para humanos.
- **Feedback Visual Deficiente:**
 - **Hallazgo:** La interfaz no notifica adecuadamente el éxito o fallo de operaciones en segundo plano (como el envío de SMS) de manera clara, limitándose a una etiqueta de estado pequeña.

3.8. Vulnerabilidades de Inyección

Aunque el uso de SQL con parámetros (`?`) previene "Inyección SQL", la falta de validación de entrada permite otros tipos de ataques graves:

- **Inyección de Delimitadores (Protocolo Serial):**
 - **Hallazgo:** La trama de datos enviada por el puerto serial (`ttyACM1`) se construye concatenando campos con comas (CSV) en el método `_generar_linea_datos` (Línea 759).
 - **Vulnerabilidad:** El código **NO filtra** comas en los campos de texto (`Identificacion` , `NumGuia`).
 - **Ejemplo de Ataque (Payload):** Si un usuario ingresa `ABC, 123` en el campo "Guía Madre".
 - **Resultado:** La trama generada tendrá campos extra indeseados, rompiendo el conteo de índices en el sistema receptor (ERP o Pantalla Remota).
 - Trama esperada: `...GUIA, TIPO, AUX...`
 - Trama inyectada: `...ABC, 123, TIPO, AUX...`
 - **Impacto:** Corrupción de datos en el destino, desplazando valores numéricos a columnas incorrectas.
- **Inyección de Fórmulas (CSV Injection - Peligro Potencial - Solo para versiones antiguas de Excel):**
 - **Riesgo Latente:** Si los datos de SQLite se exportan posteriormente a Excel/CSV para reportes gerenciales (común en ingenios).
 - **Ejemplo:** Ingresar `=cmd|'/C calc '!A0` en el campo de identificación.

- **Impacto:** Podría ejecutar código malicioso en la computadora del analista que abra el reporte en Excel.

4. Conclusión y Recomendación

El código actual representa un prototipo ("Proof of Concept") funcional pero frágil, no apto para un entorno de producción o para escalar a nuevas funcionalidades.

Se justifica la reestructuración completa para:

1. **Mantenibilidad:** Desacoplar la lógica (MVC) para permitir correcciones rápidas y seguras.
2. **Robustez:** Implementar manejo de hilos profesional y comunicación serial asíncrona o no bloqueante.
3. **Configurabilidad:** Eliminar valores "quemados" en el código.

Plan de Acción Recomendado:

1. Realizar un análisis de requerimientos actuales y planes futuros de crecimiento y escalamiento (nuevas funcionalidades, hardware, etc), con el fin de confirmar si el proyecto requiere una reestructuración completa.
2. Escoger un framework de desarrollo adecuado para el proyecto.
3. Establecer un patrón de diseño para la aplicación (se recomienda MVC).
4. Diseñar una estructura de roles y permisos acorde a las necesidades de la empresa.
5. Separar lógica de base de datos (repositorio) y hardware (drivers) de la UI.
6. Implementar una cola de tareas para el envío de SMS en lugar de crear hilos infinitos.
7. Extraer configuración a archivo de entorno.

5. Catálogo de Funcionalidades Legadas

Funcionalidad	Descripción	Entradas (Inputs)	Salidas (Outputs)
Lectura de Peso	Captura el peso estable desde el indicador de báscula vía puerto serial.	Stream de datos serial (trama continua), Puerto <code>/dev/ttyACM1</code> .	Valor numérico (float) en campo de texto UI.

Funcionalidad	Descripción	Entradas (Inputs)	Salidas (Outputs)
Registro de Pesaje	Valida y guarda la transacción de pesaje en la base de datos local.	Datos de formulario (Placa, Guía, Hacienda, Pesos), Fecha/Hora actual.	Nuevo registro en tabla SQLite, ID Consecutivo incrementado.
Cálculo de Impurezas	Calcula pesos netos y porcentajes de materia extraña (Mineral/Vegetal).	Pesos brutos y taras (Total, Mineral, Vegetal).	Valores porcentuales (%) en UI, Valores netos (kg) en DB.
Envío SMS Automático	Tarea en segundo plano que envía reportes a jefes de zona en horarios programados.	Hora del sistema, Lista <code>horas.txt</code> , Lista <code>contactos.txt</code> , Registros en DB.	Tramas PDU enviadas al Módem GSM, Logs de estado.
Carga de Maestros	Inicializa las listas desplegables de Haciendas y Suertes al arranque.	Archivo <code>Haciendas.csv</code> .	Listas en memoria <code>haciendas_list</code> , <code>suertes_dict</code> .
Edición Configuración	Editor de texto simple para modificar teléfonos y horarios.	Archivos <code>contactos.txt</code> o <code>horas.txt</code> .	Archivos de texto sobreescritos en disco.
Backup Manual	Copia física del archivo de base de datos a una ubicación externa.	Ruta de destino seleccionada por usuario.	Archivo <code>.db</code> duplicado.